



EAST TEXAS A&M
— UNIVERSITY —

CSCI 380.61E
WEB PROGRAMMING
AND INTERFACE DESIGN

OVERVIEW

- How to store **data** locally on client?
- How web browser **cookies** work?
- How to manage files and data locally?
- **Examples**
- Questions and discussion

REVISION QUESTION

Which of the following is correct to write “Hello World” on the web page?

- A. `System.out.println("Hello World");`
- B. `print("Hello World");`
- C. `document.write("Hello World");`
- D. `response.write("Hello World");`

ANSWER

Answer: **C.** `document.write("Hello World")`

REVISION QUESTION

What does RSS stand for?

- A. Runtime System Syntax
- B. Raw Standard Storage
- C. Real Syntax Standard
- D. Really Simple Syndication

ANSWER

Answer: **D.** Really Simple Syndication (RSS)

REVISION QUESTION

Javascript is an object oriented language?

- True
- False

ANSWER

Answer: **True**

Details [Link](#)

JavaScript is heavily object-based. Objects are associative arrays, augmented with prototypes (see below). Object property names are associative array keys: `obj.x = 10` and `obj["x"] = 10` are equivalent, the dot notation being merely syntactic sugar. Properties and their values can be added, changed, or deleted at run-time. The properties of an object can also be enumerated via a `for...in` loop.

REVISION QUESTION

How to append a value to an array of Javascript?

- A) `arr[arr.length] = value;`
- B) `arr[arr.length+1] = new Arrays();`
- C) `arr[arr.length-1] = value;`
- D) `arr[arr.length] = [value];`

ANSWER

Answer: **A)** `arr[arr.length] = value;`

WHATS AND WHYS OF CLIENT DATA

- **Why** would you want the **client** to store information?
- What **type** of information is stored by the client?
- What are the **legal** aspects?
- What **limitations** would there be (e.g., size)?

WEB STORAGE



WHAT IS WEB STORAGE?

WHAT IS WEB STORAGE?

- Web storage is a means by which web applications can store data **locally** within the user's **browser**
- Before HTML5 and **Web Storage** application data had to be stored in **cookies**
 - Not just cookies

WEB STORAGE

- **JavaScript** API
- Web storage is **key/value** on **web browsers**
 - stores user data locally on host running browser put it in silos correlated with web server **domain origins**
- Only web pages from domain D can access D's silo via web storage
 - **localStorage** stores **data persistently** for each web server
 - **sessionStorage** stores data **sessionally** per server per browser instance

WHY WEB STORAGE?

WHY WEB STORAGE?

- Web storage is more **secure**
- Large amounts of data can be stored locally
- Doesn't hinder website **performance**
- Web storage avoids
 - size limits of cookies
 - security issues of cookie traffic with web server

WEB STORAGE **FUNCTIONS**

Support the following functions

- **setItem(key, value)** set key to value
- **getItem(key)** get value of key
- **removeItem(key)** delete key and its value
- **clear()** remove all key/value pairs
- **key(i)** get ith element of array of keys

WEB STORAGE

- Web storage stores data **sessionally** or **persistently**
- Session data stored in browser's **volatile memory** and is lost when browser closes

HOW DO WE KNOW IF WEB STORAGE IS SUPPORTED?

IS WEB STORAGE SUPPORTED?

```
if (window.localStorage) {  
  // Browser supports localStorage  
  // Good to go  
} else {  
  // No localStorage support  
}
```

EXAMPLE (LOCAL STORAGE)

```
<form>
  <label>Name: <input type="text" id="sName"></label>
  <button type="button" onclick="savewb()">Save Name</button>
  <button type="button" onclick="accesswb()">Get Name</button>
</form>
```

```
<script>
if (localStorage) { // Check if the localStorage object exists
  function savewb() { // Store data
    var txt = document.getElementById('sName').value;
    localStorage.setItem("name", txt);
    alert("Your name is saved.");
  }

  function accesswb() { // Retrieve data
    alert("Hi, " + localStorage.getItem("name"));
  }
} else {
  alert("Sorry, your browser does not support local storage.");
}
</script>
```

EXAMPLE (SESSION STORAGE)

- **Sessional**
- Example defines **3 functions** to use session storage
 - **savewb()** updates web storage with key/value
 - **acesswb()** display keys and values in web storage
 - **clearwb()** removes key/value pairs from web storage

EXAMPLE (SESSION STORAGE)

```
<form>
  <label>String Key:   <input type="text" id="sKey"></label>
  <label>String Value: <input type="text" id="sValue"></label>
  <button type="button" onclick="savewb()" >Save </button>
  <button type="button" onclick="accesswb()">Get  </button>
  <button type="button" onclick="clearwb()" >Clear </button>
</form>
```

EXAMPLE (SESSION STORAGE)

```
function savewb() {  
    if ( typeof(sessionStorage) == "undefined" ) {  
        alert("no session web storage available");  
        return;  
    }  
    var key = document.getElementById("sKey").value;  
    if ( key != "" )  
        sessionStorage[key] = document.getElementById("sValue").value;  
}
```

```
function accesswb() {  
    var s = "";  
    for (var i=0; i<sessionStorage.length; i++) {  
        var key = sessionStorage.key(i);  
        s += key + " = " + sessionStorage.getItem(key) + "\n";  
    }  
    alert( s );  
}
```

```
function clearwb() {  
    sessionStorage.clear();  
}
```

COOKIES



WHAT ARE BROWSER COOKIES?

BROWSER COOKIES

- Cookies are **small** packets of data stored in file space of web users
- Cookies attributes:
 - **name**
 - **value**
 - domain
 - path
 - duration
- Webservers ask web clients to store cookies

COOKIES

- Cookies
 - avoid HTTP's statelessness by storing **data on client**
 - are passed in **Set-Cookie** response and **Cookie** request headers
 - are stored in user's filesystem or in browser's volatile memory
- Cookies can be **manipulated** by
 - meta directive in HTML
 - **JavaScript** in web page
 - web server programs (e.g., CGI, servlets, JSP, Node.js)

COOKIE STANDARDS

- There used to be **2 cookie standards** - Netscape's and RFC 2965.
- Combined reformed standard **RFC 6265** now governing cookie form
 - Cookie standards ([Details](#))

Optional **attributes**, which may occur once in any order, are:

- **Domain** domain of validity - defaults to server's hostname
- **Expires** date time when cookie expires
- **HttpOnly** cookie is only to be sent over HTTP
- **Max-age** how many seconds from now onwards to keep cookie
- **Path** defaults to path of document creating cookie
- **Secure** cookie is only to be sent on secure channel (HTTPS)

Max-age higher priority than Expires if both are present

COOKIE SYNTAX

- In server response header:

```
Set-Cookie: name=value [; Expires=date]
            [; Domain=domain]
            [; Path=path]
            [; Secure]
            [; HttpOnly]
```

- Date time must use 4 digit year required by RFC 1123 section 5.2.14.

EXAMPLE

```
Set-Cookie: Customer="BOB";  
            Path=/acme;  
            Max-age=31536000; Domain=dod.gov
```

- RFC 6265 example request with Max-age set (in seconds)
- It specifies cookie should last for **one year in seconds**.

COOKIE PROPERTIES

- **Session** cookies last **only** as long as client's **session**
 - have **no Max-age** or Expires setting
 - aren't stored in cookie file(s) on client
 - usually stored in browser's volatile memory
- Saving cookie with exactly same Path, Domain and Name values
 - **overwrites existing** one
 - deletes it if expiry date is past

COOKIE CONTROL

- **Path matching** is done by string comparison
 - path `"/ban"` matches `"/ban/bomb.html"` and `"/banana.html"`
 - most general path is `"/"`
 - default path is URL path of associated document
- **Browsers** often allow users to **control** whether to
 - accept all cookies
 - accept only cookies from web server's domain
 - do not accept or send cookies
 - warn and let user control acceptance of cookies
 - limit maximum lifetime of cookies

COOKIE CONTROLS

- **Browser** preferences
- Mozilla: Edit, Options > Privacy and Security > Cookies
- Edge: **Settings > Site permissions > Cookies and site data**, ...

HANDLING COOKIES IN WEB PAGES (HEADER)

- Cookie can be set by adding directive to **header of HTML page**

```
<meta http-equiv="Set-Cookie" content="name=Harry; Max-age=86400">
```

- It requests browser to act as if it had received the HTTP header

```
Set-Cookie: name=Harry; Max-age=86400
```

- It is removed with a

```
<meta http-equiv="Set-Cookie" content="name=; Max-age=-1">
```

HANDLING COOKIES IN WEB PAGES (JAVASCRIPT)

- Cookies are set in JavaScript in following manner:

```
document.cookie = "name=value; Expires=...; Path=...; Domain=...";
```

Consider the Javascript (**helper**) function

```
function Cookie(name, value, days) {  
    d=new Date();  
    if ( days < 0 ) {  
        d.setTime(0);  
    }  
    else {  
        d.setTime(d.getTime() + days * 24*60*60*1000);  
    }  
    return name + "=\"\" + value + "\"; Expires=\"\" + d.toUTCString() + "\"\";  
}
```

- **Set** a cookie for a year using

```
<body onload="go()">
<script>
function go() {
  c = Cookie("Skill", "basic", 365) + "; Domain=hw.ac.uk; Path=/";
  document.cookie = c;
  document.writeln("Cookie set: " + c); }
</script>
</body>
```

- Same cookie can be **deleted** by giving it a date in the **past**:

```
<body onload="go()">
<script>
function go() {
  c = Cookie("Skill", "anything", -1) + "; Domain=hw.ac.uk; Path=/";
  document.cookie = c;
  document.writeln("Cookie set: " + c); }
</script>
</body>
```

EXAMPLE HANDLING COOKIES IN JAVASCRIPT

Can write useful JavaScript functions for handling cookies:

- **getCookieNames()** returns array of cookie names.
- **getCookieValue(name)** takes a string argument and returns its cookie value

EXAMPLE

```
function getCookieNames() {  
    if ( document.cookie == "" ) return new Array();  
    var c = document.cookie.split(";");  
    var names = new Array(c.length);  
    for (var i=0; i<c.length; i++)  
        names[i] = c[i].substr(0, c[i].indexOf("="));  
    return names;  
}
```

```
function getCookieValue(name) {  
    var cs = document.cookie.split(";");  
    for (var k=0; k<cs.length; k++) {  
        Nam = cs[k].substr(0, cs[k].indexOf("="));  
        Val = cs[k].substr(cs[k].indexOf("=") + 1);  
        if ( Nam == name )  
            return unescape(Val);  
    }  
    return null;  
}
```

EXAMPLE

List all cookies

```
<html>
<body>
<script type="text/javascript">
var arr = getCookieNames();
for (var j=0; j<arr.length; j++) {
    var val = getCookieValue(arr[j]);
    if ( val != null ) {
        document.writeln(arr[j] + " = " + unescape(val));
        document.writeln("<br>");
    }
}
</script>
</body>
</html>
```

COOKIES, WEB STORAGE AND EU LAW

- In 2012 UK implemented clause 66 of EU directive 2009/136/EC
 - requires consent for storage or access to information
 - where it is stored on a user's equipment
- Change was motivated by concerns about
 - **online tracking** of individuals and use of spyware
 - on their devices **without their knowledge** and agreement
- Directive **applies** to use of **cookies** and use of **HTML5 Web Storage**

EXPLICIT CONSENT

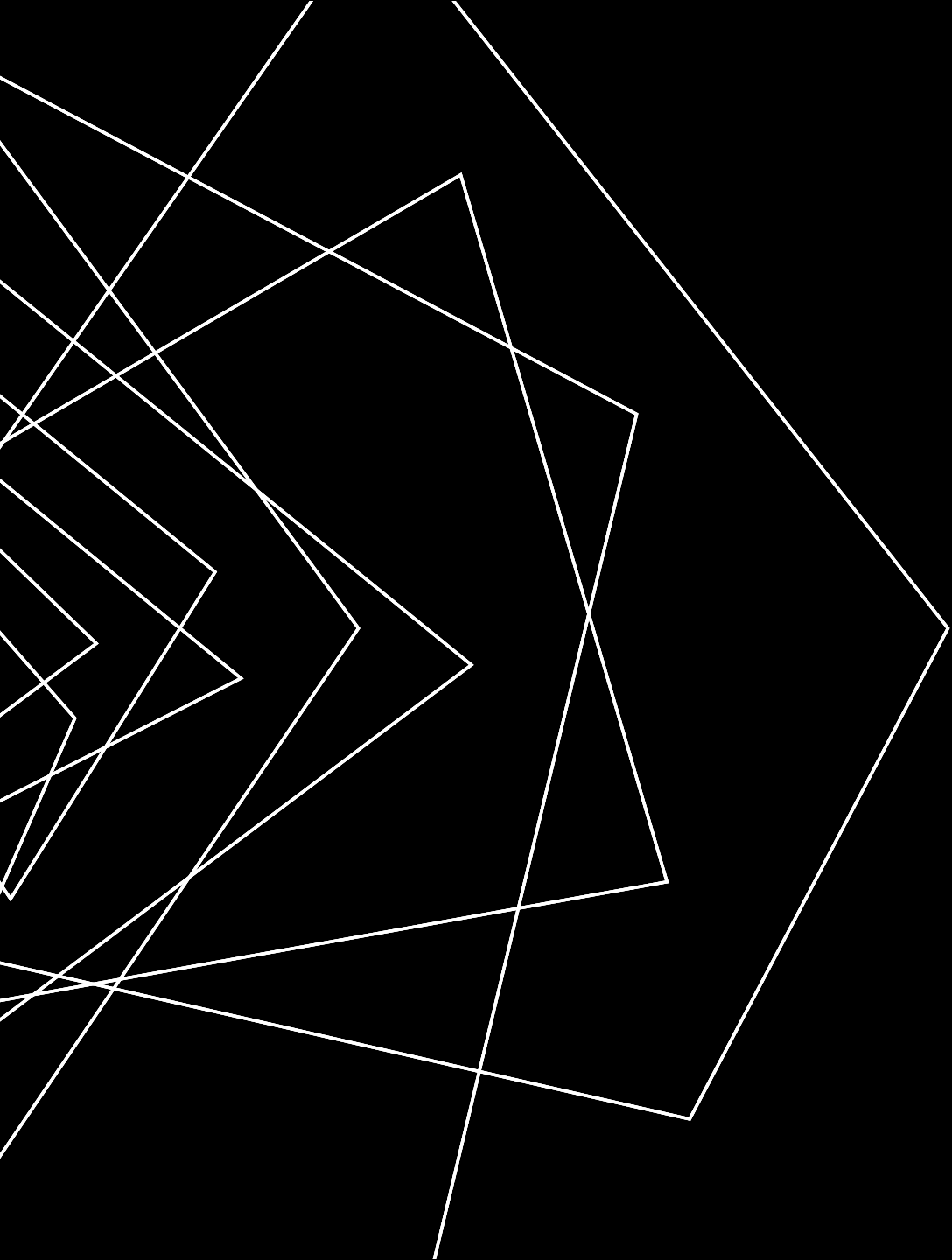
- Consent should be **explicit** and **prior** if possible
 - **freely given** specific and informed indication of a data subject's wishes by which he signifies his agreement to personal data relating to him being processed
- i.e. it must be specific, informed and express user's wishes
- Consent isn't needed where personal data storage on user computer
 - (a) is solely to enable communication over electronic network
 - (b) is strictly necessary to provide an information society service requested by user

LIMITATIONS

- Informed **consent** is **not** (supposed to be) needed for
 - a shopping cart
 - security requirement like authentication to protect user's privacy
- Needed for **advertising**, **analysing** page usage, **customisation**
- UK Information Commissioner supplies guidance on compliance

SUMMARY

- Understand Client **Data Management**
- **Web Storage** and **Cookies**
- Web storage key/value Javascript API
- Cookies name and value
- Syntax and examples
- UK **law** and **regulations**



THANK YOU

Yau Ka Cheung
Yauka.Cheung@etamu.edu
ACB1 Room 335